

Wnioskowanie statystyczne dla informatyków

Przedziały ufności, testowanie hipotez, diagnostyka i metody resamplingowe

dr hab. Andrzej Jankowski, prof. UWM

Uniwersytet Warmińsko-Mazurski w Olsztynie

20 kwietnia, 2026

Statystyka: Twoje narzędzie do debugowania rzeczywistości

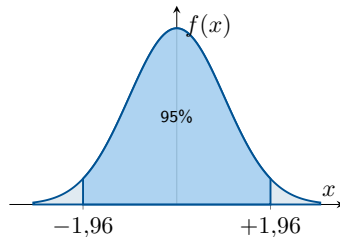
Od danych do decyzji

- Od „wydaje mi się” do „wiem z określonym prawdopodobieństwem”
- Estymacja punktowa (jedna liczba) to za mało
- Uczciwość wobec niepewności = lepsze decyzje inżynierskie

Zapowiedź

Na końcu wykładu będziesz umiał:

- skonstruować i zinterpretować przedział ufności,
- przeprowadzić test hipotezy,
- zweryfikować założenia testu,
- zbudować własny test permutacyjny.



Przykładowy wynik końcowy:

$$n = 30, \bar{x} = 92,4 \text{ ms}, 95\% \text{ CI} = [89,4; 95,4]$$

Dlaczego jedna liczba to za mało?

Estymacja punktowa kłamie w białych rękawiczkach

Problem

Z tej samej populacji dwie różne próby dają dwie różne średnie. Która jest „prawdziwa”?

```
import numpy as np
np.random.seed(42)
mu = 100
for i in range(5):
    proba = np.random.normal(mu,
                              10, size=10)
    print(f"Próba {i+1}:  $\bar{x}_{sr} = \{$   

            $\text{proba.mean():.2f}\}$ ")
```

Wynik

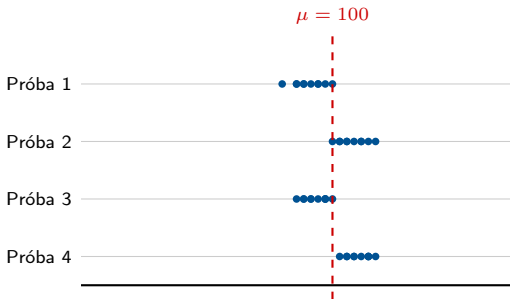
Próba 1: $\bar{x} = 96,25$

Próba 2: $\bar{x} = 102,31$

Próba 3: $\bar{x} = 98,47$

Próba 4: $\bar{x} = 103,12$

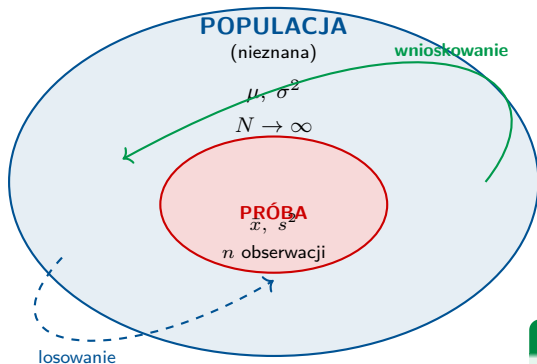
Próba 5: $\bar{x} = 95,88$



Żadna próba nie trafia dokładnie w μ

Populacja vs. próba: fundamentalne rozróżnienie

Czego naprawdę chcemy się dowiedzieć?



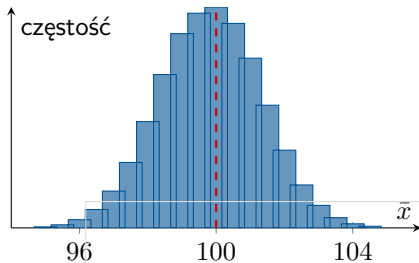
Pojęcie	Populacja	Próba
Średnia	μ	$\bar{x}$
Wariancja	σ^2	s^2
Odch. std	σ	s
Rozmiar	N	n
Status	nieznany	obliczalny
Notacja	grecka	łacińska

Puenta

Statystyka = sztuka mówienia o μ , gdy widzisz tylko $\bar{x}$.

Estymator jako zmienna losowa

Średnia z próby sama jest losowa



Histogram 10 000 średnich z niezależnych prób ($n = 30$).

Kluczowy wzór

$$SE(\bar{x}) = \frac{\sigma}{\sqrt{n}}$$

Błąd standardowy średniej.

```
srednie = [np.random.normal(100, 10, 30).  
            mean()  
            for _ in range(10000)]  
print(np.mean(srednie))      # 100.00  
print(np.std(srednie))       # 1.83  
print(10/np.sqrt(30))        # 1.83
```

Wynik

Symulacja potwierdza wzór: $\sigma/\sqrt{n} = 10/\sqrt{30} \approx 1,83$.

Centralne Twierdzenie Graniczne

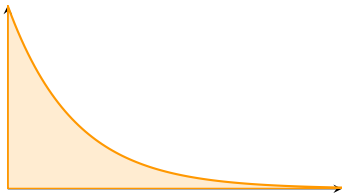
Dlaczego wszystko kończy się dzwonem Gaussa?

Twierdzenie

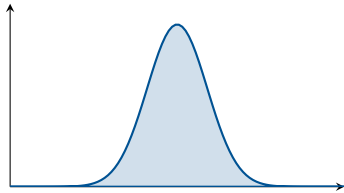
Dla dowolnego rozkładu o skończonych μ i $\sigma^2 < \infty$, dla niezależnych $X_1, \dots, X_n$:

$$\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i \xrightarrow{d} \mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right) \quad \text{przy } n \rightarrow \infty$$

Rozkład populacji (wykładniczy)



Rozkład średniej dla $n = 30$



Niezależnie od kształtu populacji, rozkład średniej zmierza do normalnego.

Centralne Twierdzenie Graniczne

Praktyczne wnioski i ograniczenia

Reguła kciuka

$n \geq 30$ zwykle wystarcza do zadziałania CTG (dla rozkładów bez ekstremalnej skośności).

Konsekwencja

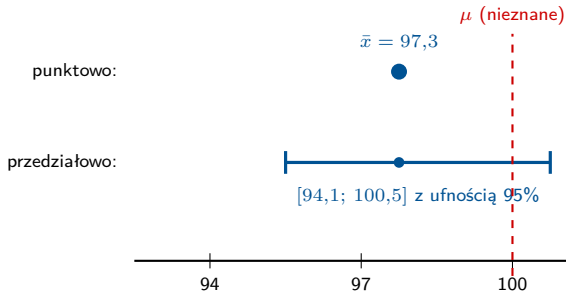
Nawet jeśli dane *nie* są normalne, ich średnia jest (dla dużych n). To dlatego testy parametryczne oparte na średniej działają tak szeroko w praktyce — wiele metod statystycznych (testy t , przedziały ufności dla średniej) korzysta z tego faktu, nawet gdy populacja nie jest gaussowska.

Ostrzeżenie

CTG *nie* działa magicznie dla rozkładów bez skończonej wariancji (np. Cauchy, niektóre rozkłady Pareto). Zawsze sprawdzaj skośność i obecność ciężkich ogonów w danych!

Przedział ufności: intuicja

Nie strzelaj punktowo. Użyj siatki.



Dwa składniki CI

- Zakres (dolna i górna granica)
- Poziom ufności (90%, 95%, 99%)

Zależność od n

Próba	Szerokość CI
$n = 10, s = 15$	21,4
$n = 100, s = 15$	6,0
$n = 1000, s = 15$	1,9

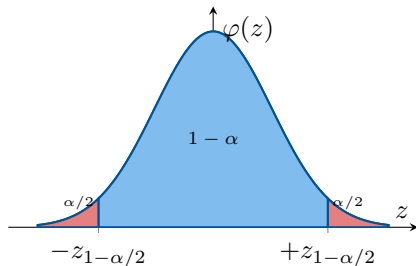
Trójkąt kompromisu

Ufność $\times$ precyzja $\times n$: **wyberz dwie**, trzeciej nie dostaniesz za darmo.

Konstrukcja CI dla średniej (σ znane) - Anatomia przedziału ufności.

Wzór fundamentalny: UWAGA: $z_{1-\alpha/2}$ to kwantyl rozkładu $\mathcal{N}(0, 1)$.

$$CI_{1-\alpha} = \bar{x} \pm z_{1-\alpha/2} \cdot \frac{\sigma}{\sqrt{n}}$$



Ufność	α	$z_{1-\alpha/2}$
90%	0,10	1,645
95%	0,05	1,96
99%	0,01	2,576

Przykład obliczeń

Dane: $n = 100$, $\bar{x} = 98,5$, $\sigma = 10$, $\alpha = 0,05$.

$$SE = \sigma / \sqrt{n} = 10 / \sqrt{100} = 1,00$$

$$ME = 1,96 \cdot 1,00 = 1,96$$

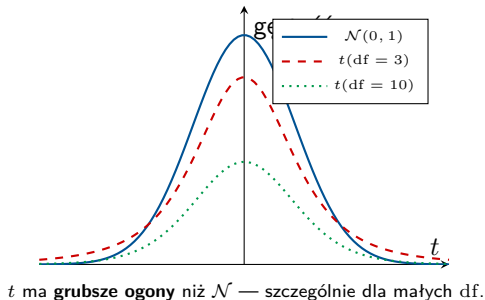
$$CI = 98,5 \pm 1,96 \\ = [96,54; 100,46]$$

Trzy składniki

Estymator punktowy + kwantyl + błąd standardowy.

CI gdy σ jest nieznane: rozkład t -Studenta

W realnym świecie NIGDY nie znamy σ



Wzór z rozkładem t

$$CI = \bar{x} \pm t_{1-\alpha/2, n-1} \cdot \frac{s}{\sqrt{n}}$$

Zmiana: $z \rightarrow t$, $\sigma \rightarrow s$, stopnie swobody
 $df = n - 1$.

df	$t_{0,975}$	vs 1,96
5	2,571	+31%
10	2,228	+14%
30	2,042	+4%
100	1,984	+1%
∞	1,960	= z

Dla małych prób: „podatek niepewności”.

Dla małych prób płacisz „podatek niepewności”. Dla $n \geq 30$ różnica zanika — CTC ratuje

Case study: CI dla czasów odpowiedzi API

Pełny rachunek krok po kroku

Dane ($n = 15$ pomiarów w ms)

94, 102, 98, 105, 91, 99, 108, 96, 101, 93, 97, 104, 100, 95, 103

Krok 1 – Średnia:

$$\bar{x} = \frac{1486}{15} = 99,07$$

Krok 2 – Wariancja próbkowa:

$$s^2 = \frac{\sum (x_i - \bar{x})^2}{n - 1} = \frac{308,93}{14} = 22,07$$

Krok 3 – Odchylenie standardowe:

$$s = \sqrt{22,07} = 4,70$$

Krok 4 – Błąd standardowy:

$$SE = s/\sqrt{n} = 4,70/\sqrt{15} = 1,21$$

Krok 5 – Stopnie swobody: $df = 14$

Krok 6 – Kwantyl:

$$t_{0,975, 14} = 2,145$$

Krok 7 – Margines błędu:

$$ME = 2,145 \cdot 1,21 = 2,60$$

Wynik: $CI_{95\%} = [96,47; 101,67]$

```
from scipy import stats
stats.t.interval(0.95, 14,
loc=99.07, scale=1.21)
# (96.47, 101.67)
```

Interpretacja CI: pułapki

Co CI *nie* znaczy

ŹLE

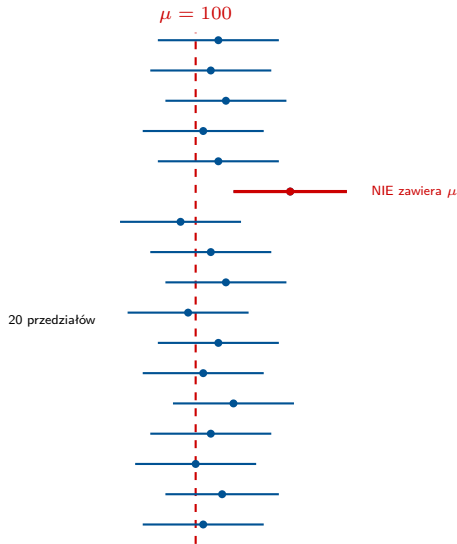
„Prawdopodobieństwo, że $\mu \in [96,47; 101,67]$ wynosi 95%.”

Dlaczego źle: μ nie jest zmienną losową. Jest **stałą** (nieznaną).

DOBRCZE

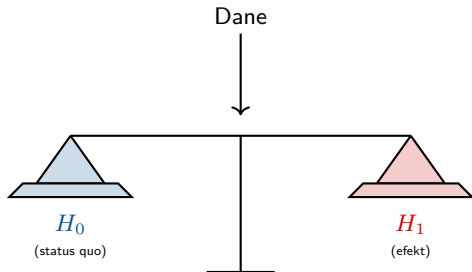
„Gdybyśmy powtórzyli eksperyment wiele razy, ok. 95% tak skonstruowanych przedziałów zawierałoby μ .”

Dlaczego dobrze: losowy jest **przedział**, nie μ .



H_0 vs H_1 : sąd nad danymi

Domniemanie niewinności dla hipotez



Sąd karny	Statystyka
Oskarżony niewinny	H_0 prawdziwa
Dowody winy	Dane
„Poza wszelką wątpliwość”	$p\text{-value} < \alpha$
„Winny”	Odrzucamy H_0
„Brak winy udowodnionej”	Nie odrzucamy H_0
NIE „niewinny”!	NIE „H_0 prawdziwa”!

Przykłady hipotez

- $H_0: \mu_A = \mu_B$ vs $H_1: \mu_A \neq \mu_B$ (A/B test)
- $H_0: t_{po} = t_{przed}$ vs $H_1: t_{po} < t_{przed}$ (refaktor)

Najczęstszy błąd

„Nie odrzucamy H_0 ” $\neq$ „ H_0 jest prawdą”. To tylko „brak dowodów”.

Statystyka testowa

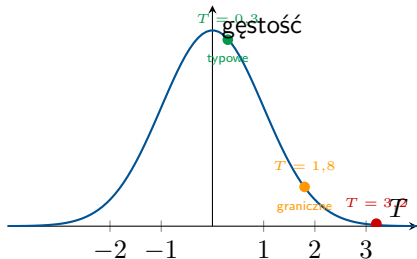
Jak przełożyć dane na jedną liczbę?

Wzór dla testu t jednej próby

$$T = \frac{\bar{x} - \mu_0}{s/\sqrt{n}}$$

Licznik: jak daleko od H_0 . **Mianownik:** skala losowego szumu.

$\Rightarrow T =$ „o ile odchyleń standardowych się różnimy”.



Dane ze slajdu 9: $\bar{x} = 99,07$, $s = 4,70$, $n = 15$.

Test $H_0 : \mu = 100$:

$$T = \frac{99,07 - 100}{4,70/\sqrt{15}} = \frac{-0,93}{1,21} = -0,77$$

$|T|$ mała $\Rightarrow$ dane zgodne z H_0 .

Reguła

$|T|$ mała $\Rightarrow$ zgodność z H_0 .

$|T|$ duża $\Rightarrow$ dowód przeciw H_0 .

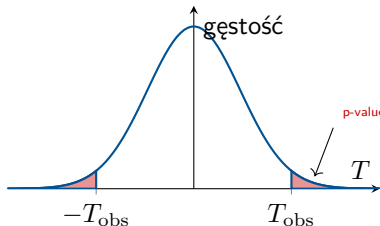
p-value: ostateczna definicja

Czy to tylko przypadek?

Definicja

$$p = \mathbb{P}(|T^*| \geq |T_{\text{obs}}| \mid H_0)$$

Prawdopodobieństwo otrzymania danych **co najmniej tak ekstremalnych** jak obserwowane, **przy założeniu**, że H_0 jest prawdziwa.



p-value	Decyzja	Dowód przeciw H_0
$< 0,001$	odrzuć	bardzo silny
$[0,001; 0,01)$	odrzuć	silny
$[0,01; 0,05)$	odrzuć	umiarkowany
$[0,05; 0,10)$	szara strefa	słaby
$\geq 0,10$	nie odrzucaj	brak

Kluczowe ostrzeżenie

p-value to **NIE** jest $\mathbb{P}(H_0 \mid \text{dane})$!

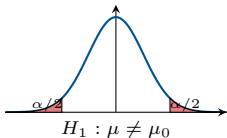
To jest $\mathbb{P}(\text{dane} \mid H_0)$.

Tych dwóch wielkości nie można ze sobą mylić!

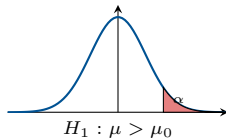
Test jednostronny vs dwustronny

W którą stronę patrzymy?

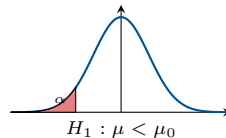
DWUSTRONNY



PRAWOSTRONNY



LEWOSTRONNY



Przykład: ta sama $T = 1,8$, $n = 20$, $df = 19$

dwustronny: $p = 2 \cdot \mathbb{P}(T > 1,8) = 0,088$

prawostronny: $p = \mathbb{P}(T > 1,8) = 0,044$

NIE odrzucamy
odrzucaamy

UWAGA

Wybór kierunku testu MUSI nastąpić **przed** zebraniem danych. Inaczej $\Rightarrow$ *p-hacking*.

Błędy I i II rodzaju

Macierz pomyłek testu hipotez

	H_0 prawdziwa	H_1 prawdziwa
Nie odrzucamy H_0	✓ POPRAWNIE $\mathbb{P} = 1 - \alpha$	X BŁĄD II RODZAJU $\mathbb{P} = \beta$ (False Negative)
Odrzucamy H_0	X BŁĄD I RODZAJU $\mathbb{P} = \alpha$ (False Positive)	✓ POPRAWNIE $\mathbb{P} = 1 - \beta = \text{MOC}$

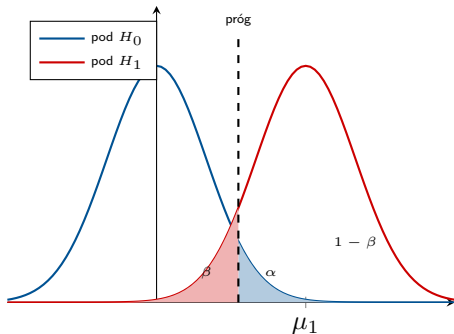
Sytuacja	Błąd I (FP)	Błąd II (FN)
Antywirus	alarm na czysty plik	przepuszcza wirusa
Testy	czerwony bez buga	zielony z bugiem
Spam	legit w spamie	spam w skrzynce
Medycyna	zdrowy → chory	chory → zdrowy

Przy 100 testach

Dla $\alpha = 0,05$, jeśli przeprowadzisz 100 niezależnych testów na identycznych zbiorach: średnio **5 z nich** da fałszywie istotny wynik.
To fundament problemu *multiple testing*.

Handel - w poszukiwaniu pragmatycznego kompromisu: $\alpha \leftrightarrow \beta$

Dwa rozkłady, jeden próg — nie możesz mieć ciastka i zjeść ciastka



Moc testu

$$\text{Moc} = 1 - \beta = \mathbb{P}(\text{odrzućenia } H_0 \mid H_1)$$

- α — fałszywy alarm (FP)
- β — przeoczenie efektu (FN)
- $1 - \beta$ — zdolność wykrycia efektu

Intuicja

Przesunięcie progu w lewo zmniejsza β , ale zwiększa α . Nie da się obu naraz.

Moc testu w praktyce

Co podkreślić, żeby test naprawdę wykrywał efekt?

Moc rośnie, gdy:

- liczność próby $n \uparrow$
- wielkość efektu $|\mu_1 - \mu_0| \uparrow$
- poziom istotności $\alpha \uparrow$
(ale kosztem większej liczby FP)
- rozrzut $\sigma \downarrow$

W praktyce

n jest jedynym parametrem, który naprawdę kontrolujesz. Efekt i σ są własnościami badanego zjawiska.

Reguła projektowa

Zanim zbierzesz dane, zaplanuj n tak, by moc $\geq 0,80$. Inaczej ryzykujesz badanie, które *nie może* wykryć efektu, nawet gdy ten istnieje.

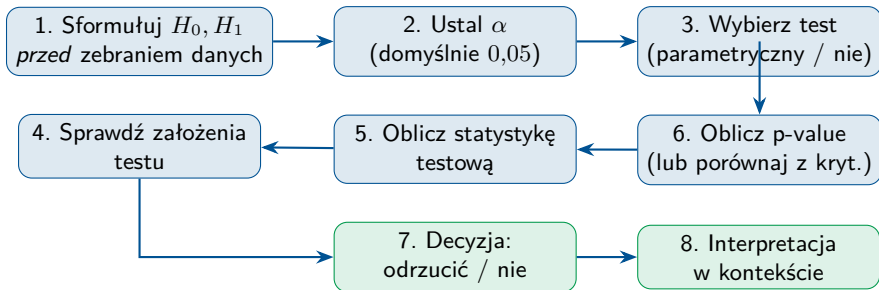
Wpływ wielkości próby na moc:

n	Moc
10	0,23
30	0,56
100	0,94

Test t , $\alpha = 0,05$, efekt = 3, $\sigma = 10$.

Procedura testu hipotez

Algorytm krok po kroku



Zasada złota

Krok 4 (założenia) jest *najczęściej pomijany* — i dlatego najwięcej błędów wnioskowania pochodzi właśnie stąd.

Case study: czy refaktor kodu pomógł?

Test t dla jednej próby — setup

Scenariusz

Przed refaktorem (niewprowadzeniem zmian w kodzie oprogramowania) średni czas odpowiedzi endpointu wynosił $\mu_0 = 100$ ms (dane historyczne, duża próba $\rightarrow$ traktujemy jako wartość znaną). Po refaktorze zmierzono **20 nowych zapytań**.

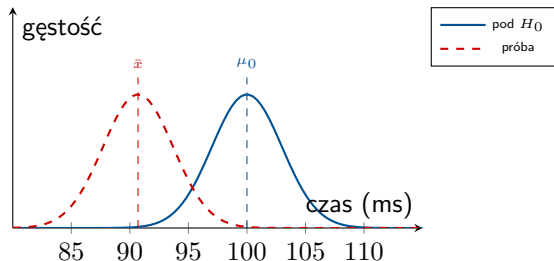
Pytanie: czy refaktor istotnie skrócił czas odpowiedzi?

Dane (ms):

92	88	95	91	89	94	87	93	90	96
85	92	89	91	88	94	90	86	93	91

Hipotezy

- $H_0: \mu = 100$ (refaktor nie pomógł)
- $H_1: \mu < 100$ (refaktor skrócił czas)
- $\alpha = 0,05$ (jednostronny)



Case study: obliczenia

Krok 1 – Statystyki opisowe:

- $n = 20, \sum x_i = 1814$
- $\bar{x} = 1814/20 = 90,70$ ms
- $s^2 = 179,80/19 = 9,463$
- $s = 3,076$ ms

Krok 2 – Błąd standardowy:

$$SE = s/\sqrt{n} = 3,076/\sqrt{20} = 0,688$$

Krok 3 – Statystyka testowa:

$$T = \frac{90,70 - 100}{0,688} = \frac{-9,30}{0,688} = -13,52$$

Krok 4 – Wartość krytyczna i p-value:

$$t_{\text{kryt}} = -1,729 \quad (\text{df} = 19, \alpha = 0,05, \text{lewostr.})$$

$$p = \mathbb{P}(T < -13,52 | H_0) \approx 1,5 \cdot 10^{-11}$$

Decyzja

$$|T| = 13,52 \gg 1,729$$

$$p \ll \alpha = 0,05$$

$\Rightarrow$ **odrzucaamy** H_0

(bardzo silny dowód)

```
from scipy import stats
dane = [92,88,95,91,89,94,87,93,90,96,
85,92,89,91,88,94,90,86,93,91]
t, p = stats.ttest_1samp(
dane, popmean=100,
alternative='less')
print(f"T = {t:.2f}")
print(f"p = {p:.2e}")
# T = -13.52
# p = 1.49e-11
```

Istotność statystyczna vs praktyczna

Cohen's d i pełne raportowanie

Wielkość efektu (Cohen's d)

$$d = \frac{\bar{x} - \mu_0}{s} = \frac{90,70 - 100}{3,076} = -3,02$$

$ d $	Interpretacja
0,2	mały
0,5	średni
0,8	duży
3,02	ogromny

Pełny raport

„Czas odpowiedzi po refaktorze ($M = 90,7$, $SD = 3,1$, $n = 20$) był istotnie niższy niż wartość bazowa 100 ms: $t(19) = -13,52$, $p < 0,001$, $d = -3,02$, 95% CI = [89,26; 92,14].”

Dla dużych prób: Przy $n = 100\,000$ efekt rzędu 0,5 ms da $p < 0,05$ — ale biznesowo jest bez znaczenia.

Zawsze raportuj **wielkość efektu**, nie tylko p-value.

Raportuj trzy rzeczy

statystyka testowa + p-value + CI & wielkość efektu

Założenie IID: cichy zabójca testów

Skąd wiemy, że obserwacje są niezależne?

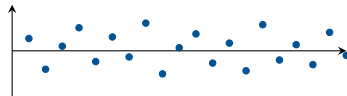
Fundamentalne założenie

Większość testów parametrycznych zakłada, że obserwacje są **niezależne i mają identyczny rozkład** (IID — *independent and identically distributed*).

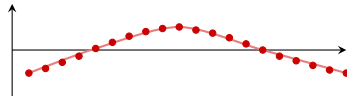
Co łamie IID w informatyce?

Źródło problemu	Przykład
Autokorelacja czasowa	latencja serwera („rozgrzewa się”)
Efekt kolejki	benchmark pod obciążeniem
Powtarzalni użytkownicy	ten sam user w A/B teście
Efekt dnia tygodnia	metryki użytkowania
Dryf systematyczny	degradacja sprzętu, cache

Losowe (IID)



Skorelowane (nie-IID)



Konsekwencje złamania IID

Zaniżony błąd standardowy $\Rightarrow$ zbyt wąskie CI $\Rightarrow$ **zaniżone p-value** $\Rightarrow$ zbyt wiele fałszywie istotnych wyników. Twoja statystyka jest pewniejsza niż w rzeczywistości.

Test serii (Wald-Wolfowitz runs test) — koncepcja (1/2)

Czy sekwencja obserwacji jest losowa?

Idea

Kodujemy każdą obserwację jako + (powyżej mediany) lub - (poniżej), i liczymy **serie** — maksymalne ciągi tego samego znaku.

Sekwencja:	+	+	+	-	-	+	+	-	+	+	-	-	-
Seria:		1			2		3		4		5		6

Liczba serii: $R = 6$

Liczba wartości powyżej mediany: $n_1 = 7$

Liczba wartości poniżej mediany: $n_2 = 6$

Interpretacja liczby serii

Za mało (klastrowanie) $\leftarrow$ **Losowo** $\checkmark$ $\rightarrow$ **Za dużo** (nadmiarowa alternacja)

Test serii (Wald-Wolfowitz runs test) — statystyka (2/2)

Weryfikacja hipotezy o losowości

Statystyka testowa pod H_0

Przy założeniu losowości, rozkład liczby serii R ma parametry:

$$\mu_R = \frac{2n_1n_2}{n_1 + n_2} + 1$$

$$\sigma_R^2 = \frac{2n_1n_2(2n_1n_2 - n_1 - n_2)}{(n_1 + n_2)^2(n_1 + n_2 - 1)}$$

Dla dużych próbek ($n_1, n_2 > 10$) stosujemy przybliżenie rozkładem normalnym:

Asymptotyczny test Z

$$Z = \frac{R - \mu_R}{\sigma_R} \xrightarrow{d} \mathcal{N}(0, 1)$$

Hipotezę H_0 odrzucamy, gdy $|Z| > z_{\alpha/2}$.

Test serii — pełny rachunek i kod

Dla przykładu z poprzedniego slajdu:

$$\mu_R = \frac{2 \cdot 7 \cdot 6}{13} + 1 = \frac{84}{13} + 1 \\ \approx 7,46$$

$$\sigma_R^2 = \frac{2 \cdot 7 \cdot 6 \cdot (84 - 13)}{13^2 \cdot 12} \\ = \frac{5964}{2028} \approx 2,94$$

$$\sigma_R \approx 1,71$$

$$Z = \frac{6 - 7,46}{1,71} = -0,85$$

```
from statsmodels.sandbox.stats.runs \
import runtest_1samp

dane = [...]
z, p = runtest_1samp(
dane, cutoff='median')
print(f"Z = {z:.2f}, p = {p:.3f}")
# Z = -0.85, p = 0.396
```

Puenta

Runs test to najprostsza broń przeciw **autokorelacji**.
Wykonaj go zanim zaufaszmy testowi t .

Decyzja

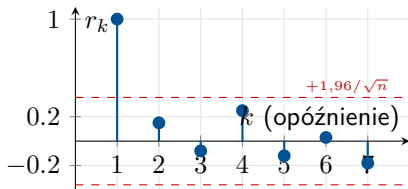
$|Z| = 0,85 < 1,96 \Rightarrow$ brak dowodów przeciw losowości ✓

Autokorelacja i test Ljung-Boxa

Ilościowe mierzenie zależności między sąsiadami

Autokorelacja przy opóźnieniu k

$$r_k = \frac{\sum_{t=k+1}^n (x_t - \bar{x})(x_{t-k} - \bar{x})}{\sum_{t=1}^n (x_t - \bar{x})^2}$$



Correlogram — wszystkie r_k w pasie ufności $\Rightarrow$ brak autokorelacji.

Test Ljung-Boxa

Globalny test dla wielu opóźnień:

$$Q = n(n+2) \sum_{k=1}^h \frac{r_k^2}{n-k} \sim \chi_h^2 \text{ pod } H_0$$

- H_0 : dane są nieskorelowane (biały szum)
- H_1 : istnieje istotna autokorelacja

```
from statsmodels.stats.diagnostic \
import acorr_ljungbox
wynik = acorr_ljungbox(dane,
lags=[5, 10], return_df=True)
```

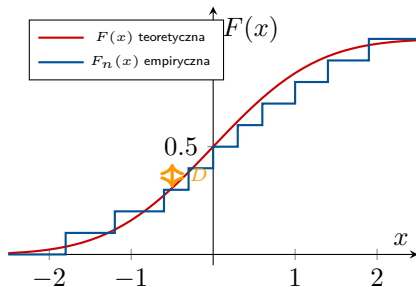
Dystrybuanta empiryczna (ECDF)

Fundament testu Kołmogorowa-Smirnowa

Definicja

$$F_n(x) = \frac{1}{n} \sum_{i=1}^n \mathbb{I}\{X_i \leq x\}$$

Ułamek obserwacji $\leq x$. Dla dużych n : $F_n(x) \rightarrow F(x)$ (prawo wielkich liczb).



Nierówność DKW

Dvoretzky-Kiefer-Wolfowitz:

$$\mathbb{P}\left(\sup_x |F_n(x) - F(x)| > \varepsilon\right) \leq 2e^{-2n\varepsilon^2}$$

Różnica maleje jak $1/\sqrt{n}$.

```
import numpy as np
def ecdf(data):
    x = np.sort(data)
    y = np.arange(1, len(x)+1)/len(x)
```

Test Kołmogorowa-Smirnowa (jedna próba)

Porównanie rozkładu empirycznego z teoretycznym

Hipotezy

- H_0 : dane pochodzą z rozkładu F_0
- H_1 : dane pochodzą z innego rozkładu

Statystyka testowa

$$D_n = \sup_x |F_n(x) - F_0(x)|$$

Maksymalna **pionowa** odległość między ECDF a hipotetyczną CDF.

Wartości krytyczne ($\alpha = 0,05$):

n	D_{kryt}	n	D_{kryt}
10	0,409	50	0,188
20	0,294	100	0,134
30	0,242	∞	$1,358/\sqrt{n}$

```
from scipy.stats import kstest
np.random.seed(0)
```

```
# Dane z  $N(0,1)$  - powinno nie odrzucić  $H_0$ 
dane = np.random.normal(0, 1, 50)
D, p = kstest(dane, 'norm', args=(0, 1))
print(f"D={D:.3f}, p={p:.3f}")
# D=0.099, p=0.693 ---> OK
```

```
# Dane z  $t(df=3)$  - grube ogony
dane2 = np.random.standard_t(df=3, size=50)
D, p = kstest(dane2, 'norm', args=(0, 1))
print(f"D={D:.3f}, p={p:.3f}")
# D=0.191, p=0.042 ---> odrzucamy  $H_0$ 
```

Zaleta

K-S jest **uniwersalny** — testuje dopasowanie do *dowolnej* hipotezy F_0 (normalny, wykładniczy, jednostajny, ...).

K-S dwupróbkowy

Czy dwie próby pochodzą z tego samego rozkładu?

Hipotezy i statystyka

- $H_0: F_X = F_Y$
- $H_1: F_X \neq F_Y$

$$D_{n,m} = \sup_x |F_n(x) - G_m(x)|$$

Ogromna zaleta

Wykrywa różnice w **całym rozkładzie** — nie tylko w średniej.
Różnica w wariancji, skośności, kurtozie — wszystko złapie.

Zero założeń

Nie wymaga żadnych założeń o rozkładach.

```
from scipy.stats import ks_2samp
```

```
# Czasy dwóch algorytmów sortowania  
A = [1.2, 1.5, 1.8, 1.3, 1.7,  
     1.4, 1.6, 1.2, 1.5, 1.8]  
B = [1.9, 2.1, 1.7, 2.3, 2.0,  
     1.8, 2.2, 1.9, 2.1, 2.0]
```

```
D, p = ks_2samp(A, B)  
print(f"D = {D:.3f}, p = {p:.4f}")  
# D = 0.800, p = 0.002  
# ---> odrzucamy H_0  
# Rozkłady są istotnie różne
```

Gdy używać?

Gdy **nie chcesz zakładać** normalności, K-S dwupróbkowy jest pierwszym wyborem.

K-S dla normalności: pułapka Lillieforsa

Ostrzeżenie: zwykły K-S *nie* nadaje się do testowania normalności

Problem

Zwykły test K-S zakłada, że F_0 jest **w pełni wyspecyfikowana**: znamy μ i σ . Gdy estymujemy μ , σ z **tej samej próby**, rozkład D pod H_0 jest *inny* (węższy). $\Rightarrow$ Zawyżone p-value $\Rightarrow$ za rzadko odrzucamy $H_0 \Rightarrow$ za łatwo uznajemy dane za normalne.

Rozwiązanie — poprawka Lillieforsa (1967)

Zmodyfikowana tablica wartości krytycznych, uwzględniająca fakt estymacji parametrów. Lilliefors jest **bardziej rygorystyczny** — wymaga mniejszego D , by odrzucić H_0 .

Wartości krytyczne ($\alpha = 0,05$):

n	K-S „naiwny”	Lilliefors	Redukcja
10	0,409	0,258	−37%
20	0,294	0,190	−35%
30	0,242	0,161	−33%
50	0,188	0,124	−34%

```
from statsmodels.stats.diagnostic \
import lilliefors
```

```
dane = [92,88,95,91,89,94,87,93,90,96,
85,92,89,91,88,94,90,86,93,91]
```

```
D, p = lilliefors(dane, dist='norm')
print(f"D = {D:.3f}, p = {p:.4f}")
# D = 0.112, p = 0.683
# ---> nie odrzucamy H_0 o normalności
```

Zasada praktyczna

Do testowania **normalności** używaj **Lillieforsa** albo **Shapiro-Wilka**, *nigdy* zwykłego K-S.

Porównanie testów normalności

Który test wybrać?

Test	Idea	Moc	Kiedy używać
Shapiro-Wilk	korelacja danych z teoretycznymi kwantylami $\mathcal{N}$	najwyższa dla $n \leq 50$	domyślny wybór dla małych/średnich prób
Lilliefors	K-S z poprawką na estymowane parametry	średnia	uniwersalność K-S dla normalności
Anderson-Darling	waży odchylenia w ogonach	wysoka	gdy podejrzewamy problem z ogonami
Jarque-Bera	bazuje na skośności i kurtozie	średnia	dla dużych prób ($n > 2000$)
K-S (naiwny)	$\sup F_n - F_0 $	NISKA dla normalności	<i>nie używać</i> dla normalności

Reguła kciuka

- $n < 50 \rightarrow$ Shapiro-Wilk
- $50 \leq n < 2000 \rightarrow$ Shapiro-Wilk + Anderson-Darling
- $n \geq 2000 \rightarrow$ **QQ-plot** (testy nadwrażliwe)

Paradoks dużych prób

Dla $n = 10\,000$ nawet mikroskopijne odchylenia od normalności dadzą $p < 0,001$. To *nie* znaczy, że testów parametrycznych nie można użyć — patrz na **wielkość odchylenia** (QQ-plot), nie tylko na p-value.

Jak powstaje wykres kwantylowy (QQ-plot)?

Porównanie rozkładu empirycznego z teoretycznym (np. normalnym)

- 1 **Sortowanie:** Uporządkuj n obserwacji z próby od najmniejszej do największej:
 $x_{(1)} \leq x_{(2)} \leq \dots \leq x_{(n)}.$
- 2 **Prawdopodobieństwo empiryczne:** Każdej obserwacji przypisz rangę i i oblicz jej dystrybuantę empiryczną, np. $P_i = \frac{i-0.5}{n}.$
- 3 **Kwantyle teoretyczne:** Dla każdego P_i oblicz, jaką wartość przyjąłaby zmienna w rozkładzie wzorcowym (np. $\mathcal{N}(0, 1)$):

$$q_i = \Phi^{-1}(P_i)$$

- 4 **Mapowanie:** Nanieś na wykres punkty o współrzędnych:

$$\text{Punkt}_i = (q_i, x_{(i)}) = (\text{kwantyl teoretyczny}, \text{kwantyl z próby})$$

Interpretacja wizualna

Jeśli punkty układają się wzdłuż linii diagonalnej $y = x$, dane pochodzą z badanego rozkładu. Odchylenia (kształt S lub U) sygnalizują inne parametry skupienia lub „grube ogony”.

QQ-plot: konstrukcja matematyczna — dane

Etap wyznaczania współrzędnych punktów

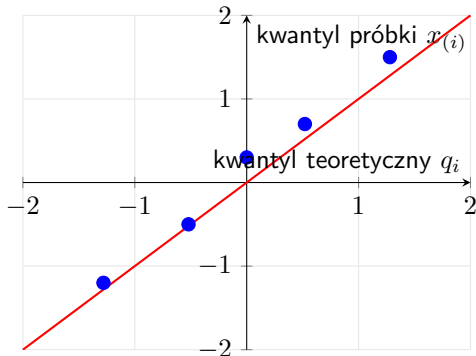
Algorytm konstrukcji

- 1 Posortuj dane: $x_{(1)} \leq x_{(2)} \leq \dots \leq x_{(n)}$
- 2 Oblicz prawdopodobieństwa empiryczne: $p_i = (i - 0,5)/n$
- 3 Wyznacz kwantyle teoretyczne: $q_i = F_0^{-1}(p_i)$

Dane	Sortuj ($x_{(i)}$)	p_i	q_i (z $\mathcal{N}$)	Punkt QQ ($q_i, x_{(i)}$)
0,7	-1,2	0,10	-1,28	(-1,28; -1,2)
-1,2	-0,5	0,30	-0,52	(-0,52; -0,5)
1,5	0,3	0,50	0,00	(0,00; 0,3)
0,3	0,7	0,70	0,52	(0,52; 0,7)
-0,5	1,5	0,90	1,28	(1,28; 1,5)

QQ-plot: konstrukcja matematyczna — wykres

Graficzna weryfikacja zgodności z rozkładem



Interpretacja

Punkty $(q_i, x_{(i)})$ naniesione na wykres powinny układać się wzdłuż linii $y = x$.

Diagnoza

W tym przykładzie punkty leżą blisko linii, co sugeruje, że dane pochodzą z rozkładu normalnego (zgodnie z założeniem F_0).

QQ-plot: implementacja i warianty

Uzupełnienie diagnostyki

Interpretacja wizualna

Punkty na linii $\implies X \sim F_0$.

- **Wygięcie w łuk:** asymetria.
- **Kształt S:** różnice w kurtozie (grube ogony).

Wybór prawdopodobieństwa p_i

Różne konwencje (*plotting positions*):

- $(i - 0,5)/n$ — standard (Continuity correction)
- $i/(n + 1)$ — Weibull
- $(i - 3/8)/(n + 1/4)$ — Blom (częsty w $\mathcal{N}$)

Implementacja (Python)

```
import scipy.stats as stats
import matplotlib.pyplot as plt

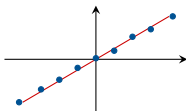
# Generowanie wykresu
stats.probplot(dane, dist='norm', plot=plt)
plt.title("Wykres QQ")
plt.show()
```

Uwaga: Pakiety statystyczne (np. `scipy`) domyślnie dopasowują linię regresji, co pozwala na testowanie przynależności do *rodziny* rozkładów (niezależnie od μ i σ).

Diagnostyka odchyleń: galeria wzorców

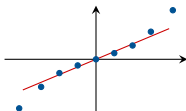
Co mówi kształt krzywej QQ?

NORMALNE



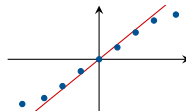
Idealnie na linii.

GRUBE OGONY



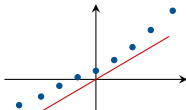
t -Studenta, Cauchy.

CIENKIE OGONY



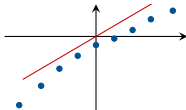
Jednostajny.

SKOŚNY PRAWOSTRONNIE



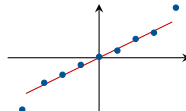
Długi prawy ogon.

SKOŚNY LEWOSTRONNIE



Długi lewy ogon.

OUTLIERY



Punkty odstające.

Puenta

Shapiro/Lilliefors mówi TAK/NIE. QQ-plot mówi **DLACZEGO** i **GDZIE**. Używaj obu.

Testy permutacyjne: mechanika działania (1/2)

Idea wymienialności etykiet (*exchangeability*)

Główna idea

Jeśli H_0 jest prawdziwa (grupy są identyczne), to przypisanie obserwacji do grup A i B jest czysto losowe. Możemy je dowolnie „przetasować” — obserwowana statystyka powinna być typowa dla takich przetasowań.

	Grupa A	Grupa B	Różnica średnich (Δ)
ORYGINAŁ:	5, 7, 9, 11, 13	2, 4, 6, 8, 10	$\Delta_{\text{obs}} = 3,0$
Permutacja 1:	7, 4, 11, 8, 13	9, 2, 5, 6, 10	$\Delta_1 = 2,2$
Permutacja 2:	2, 7, 13, 8, 10	5, 4, 9, 11, 6	$\Delta_2 = -1,0$
Permutacja 3:	5, 2, 4, 11, 6	7, 9, 13, 8, 10	$\Delta_3 = -3,8$

- Powtarzamy proces wielokrotnie (np. $N = 10\,000$ razy).
- Budujemy **empiryczny rozkład statystyki** pod H_0 .
- **Pytanie:** Jaki procent symulowanych Δ jest $\geq \Delta_{\text{obs}}$?

Testy permutacyjne: dlaczego to działa? (2/2)

Zalety, wady i puenta

Dlaczego to genialne?

- **Zero założeń** o rozkładach (brak wymogu normalności czy jednorodności wariancji).
- Działa dla **dowolnej statystyki** (różnica median, wariancji, współczynniki korelacji).
- **Precyzyjne p-value**: Wynik to po prostu frakcja permutacji bardziej ekstremalnych niż nasze dane.

Puenta

Permutacje zamieniają pytanie: „*Jaki jest teoretyczny rozkład T ?*” na zadanie: „*Wygeneruj ten rozkład samemu*”.

Wyzwania

Koszt obliczeniowy: Dla $n = 10$ w każdej grupie mamy $\binom{20}{10} = 184\,756$ permutacji. Przy dużych n stosujemy **metodę Monte Carlo** (losujemy podzbiór permutacji).

W erze wektoryzacji (NumPy/Pandas) i GPU bariera obliczeniowa praktycznie znika.

Algorytm testu permutacyjnego

Implementacja krok po kroku

Pseudokod (dla dwóch prób, statystyka = różnica średnich)

- ❶ Połącz obie próby w jeden wektor Z długości $N = n_A + n_B$
- ❷ Oblicz obserwowaną statystykę: $T_{\text{obs}} = \bar{x}_A - \bar{x}_B$
- ❸ Powtórz B razy (typowo $B = 10\,000$):
 - a. Wygeneruj losową permutację wektora Z
 - b. Pierwsze n_A elementów $\rightarrow$ grupa A^* , reszta $\rightarrow B^*$
 - c. Oblicz $T^* = \bar{x}_{A^*} - \bar{x}_{B^*}$
 - d. Jeśli $|T^*| \geq |T_{\text{obs}}|$: zwiększ licznik
- ❹ $p\text{-value} = \frac{\text{licznik} + 1}{B + 1}$

```
import numpy as np

def permutation_test(A, B, n_iter=10000,
                    statistic=np.mean):
    obs = statistic(A) - statistic(B)
    combined = np.concatenate([A, B])
    n_A = len(A)
    count = 0
    for _ in range(n_iter):
```

Dlaczego +1 w liczniku?

Trik dla testów Monte Carlo — zapobiega $p = 0$, które jest niemożliwe (oryginalna permutacja jest jedną z możliwych).

B

Dokładność n

Przykład 1: Test permutacyjny vs test t

Porównanie czasów odpowiedzi dwóch serwerów

Dane

Serwer A ($n = 8$) 12,1, 11,8, 13,2, 12,5, 11,9, 12,8, 13,0, 12,3

Serwer B ($n = 8$) 14,2, 15,1, 13,8, 14,9, 15,3, 14,5, 14,8, 15,0

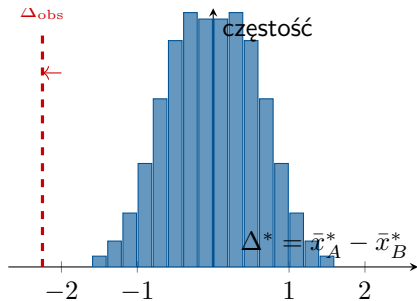
$\bar{x}_A = 12,45$, $\bar{x}_B = 14,70$, obserwowana różnica: $\Delta_{\text{obs}} = -2,25$ ms

```
import numpy as np
np.random.seed(1)

A = np.array([12.1, 11.8, 13.2, 12.5,
              11.9, 12.8, 13.0, 12.3])
B = np.array([14.2, 15.1, 13.8, 14.9,
              15.3, 14.5, 14.8, 15.0])

obs = A.mean() - B.mean() # -2.25
combined = np.concatenate([A, B])
n_A = len(A)
```

```
deltas = []
for _ in range(10000):
    np.random.shuffle(combined)
    deltas.append(combined[:n_A].mean())
```



Histogram 10000 permutacyjnych Δ^* . Żaden nie dorównuje $|\Delta_{\text{obs}}|$.

Przykład 1 c.d.: Porównanie z testem t

```
# Test permutacyjny
p_perm = 0.00030

# Test t (parametryczny)
from scipy.stats import ttest_ind
t, p_t = ttest_ind(A, B)
print(f"t-test p = {p_t:.5f}")
# t-test p = 0.00002
```

Metoda	p-value
Test permutacyjny	0,00030
Test t Studenta	0,00002
Decyzja:	obie odrzucają H_0

Wyniki zgodne

Gdy założenia testu t są spełnione, permutacja i t -test dają bardzo podobne wyniki.

Zalety permutacji

Wynik byłby tak samo wiarygodny nawet, gdyby dane były:

- wysoce skośne,
- z grubymi ogonami,
- miały bardzo małą próbę ($n < 10$),
- miały różne wariancje w grupach.

Zasada

Gdy oba testy działają — zwracają podobne p . Gdy założenia t -testu padają — tylko permutacyjny nadal działa.

Przykład 2: Test permutacyjny dla korelacji

Czy te zmienne są *naprawdę* skorelowane?

Scenariusz

Mamy dane (X, Y) — czas kompilacji vs liczba linii kodu. Obserwujemy $r_{\text{obs}} = 0,993$ ($n = 15$). Czy to istotna korelacja, czy przypadek?

Idea:

Pod H_0 (brak korelacji), X i Y są niezależne. Etykiety Y mogą być przypisane do dowolnego X . Tasujemy **tylko jedną** ze zmiennych i liczymy korelację za każdym razem.

```
import numpy as np
from scipy.stats import pearsonr
```

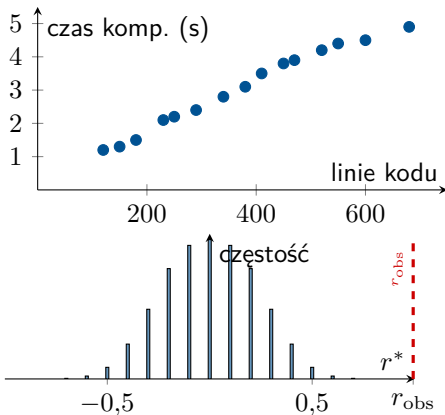
```
X = np.array([120, 450, 230, 680, 180,
              520, 340, 410, 290, 600,
              150, 380, 470, 250, 550])
Y = np.array([1.2, 3.8, 2.1, 4.9, 1.5,
              4.2, 2.8, 3.5, 2.4, 4.5,
              1.3, 3.1, 3.9, 2.2, 4.4])
```

```
r_obs = pearsonr(X, Y)[0]      # 0.993
```

```
n_perm = 10000
```

```
count = 0
```

```
Y_s = Y.copy()
```



Test permutacyjny: kiedy króluje?

Sytuacje, w których permutacja bije testy parametryczne

1. Małe próby ($n < 20$)

Testy parametryczne polegają na CTG. Permutacja jest dokładna dla każdego n .

2. „Dziwne” statystyki

Porównanie:

- **median**, kwartyli
- **maximów, zakresu**
- indeksu Giniego
- własnej miary zależności (np. MI)

Bez wyprowadzania rozkładu.

3. Heteroskedastyczność

Różne wariancje w grupach. Test t zakłada równość, permutacja — nie.

4. Dane ordinalne

Rangi, porządek — permutacja działa równie dobrze jak na liczbach.

5. Złożone eksperymenty

- ANOVA permutacyjna
- regresja permutacyjna
- GLM i modele mieszane

Sytuacja	Param.	Perm.
n duże, normalne	✓	✓
n małe, skośne	X	✓
Niestand. statystyka	X	✓
Brak teorii rozkładu	X	✓
Big Data, krótki czas	✓	△

Test permutacyjny vs bootstrap: nie myl!

Dwie krewniaczki, różne cele

Aspekt	Test permutacyjny	Bootstrap
Cel	p-value (testowanie)	CI (estymacja)
Próbkowanie	bez zwracania	ze zwracaniem
Hipoteza	H_0 wbudowana	brak H_0
Dostajemy	rozkład T pod H_0	rozkład T wokół $\hat{T}$
Typowa liczba iteracji	10 000	1 000–10 000

```
stat=np.mean, alpha=0.05):
estimates = [
stat(np.random.choice(
data, len(data), replace=True))
for _ in range(n_iter)
]
lo = 100 * alpha/2
hi = 100 * (1 - alpha/2)
return np.percentile(estimates,
[lo, hi])
```

```
dane = [94,102,98,105,91,99,108,96,
101,93,97,104,100,95,103]
```

```
ci = bootstrap_ci(dane)
```

```
# [96.87, 101.20]
```

Kiedy co?

- Testujesz H_0 ? → **permutacja**
- Chcesz CI dla parametru? → **bootstrap**
- Oba pytania? → użyj **obu**

Przykład komplementarności

- 1 Bootstrap → CI dla mediany dwóch grup
- 2 Permutacja → test $H_0: \text{med}_A = \text{med}_B$

Rozszerzona lista kontrolna

Kompletny workflow wnioskowania

FAZA 1: Planowanie

- Zdefiniuj H_0 i H_1 *przed* zebraniem danych
- Wybierz kierunek testu
- Ustal α (domyślnie 0,05)
- Oszacuj n dla mocy $\geq 0,80$

FAZA 2: Zbieranie i EDA

- Zbierz dane (randomizacja!)
- Wizualizacje (histogram, boxplot)
- Opisowe statystyki

FAZA 3: Diagnostyka

Losowość/IID:

- runs test
- Ljung-Box

Rozkład:

- Shapiro-Wilk LUB Lilliefors
- QQ-plot z pasami ufności

Homoskedastyczność:

- Levene, Bartlett

FAZA 4: Wybór testu

- założenia OK $\rightarrow$ parametr.
- złamane $\rightarrow$ nieparametr. LUB permutacja

FAZA 5: Obliczenia

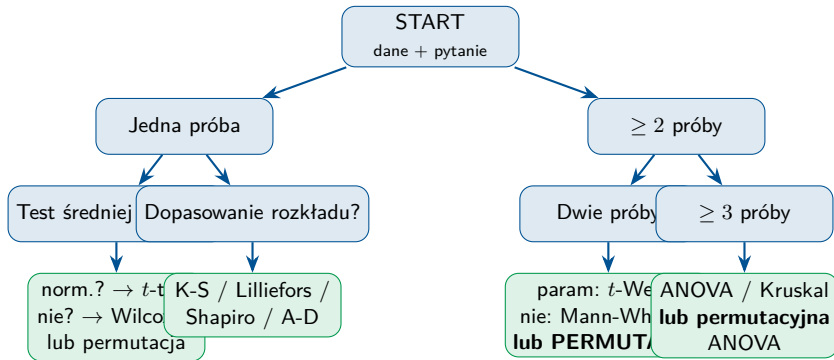
- Statystyka testowa
- p-value + CI
- Wielkość efektu (d , r , η^2)
- Decyzja vs α

FAZA 6: Raportowanie

- Pełny raport:
stat. + df + p + CI + d + n
- Interpretacja w kontekście praktycznym
- Uczciwość: opisz sprawdzone i niespełnione założenia

Drzewo decyzyjne: jaki test wybrać?

Szybka mapa decyzyjna



Rekomendacja domyślna dla informatyka: W razie wątpliwości — **test permutacyjny**.

Działa prawie zawsze, wymaga najmniej założeń, łatwo zakodować w 20 liniach Pythona.

Ostatnia myśl

Statystyka to nie magia — to *dyscyplina*. A dyscyplina polega na sprawdzaniu założeń.

Wnioskowanie statystyczne dla informatyków

Przedziały ufności, testowanie hipotez, diagnostyka i metody resamplingowe

dr hab. Andrzej Jankowski, prof. UWM

Uniwersytet Warmińsko-Mazurski w Olsztynie

20 kwietnia, 2026